NODE.JS ES MODULES | CLEAN ARCHITECTURE | MODULAR MONOLITH | PRISMA 6 + MYSQL

SHARE2GO Backend Architecture

Complete Folder Structure, Domain Separation, and Technical Specification Guide

APPLICATION	BASE URL	PROTOCOL	TARGET PLATFORMS
SHARE2GO API	/api/v1	HTTP/REST + WSS (Socket.io)	Mobile App + Web Admin

1. Executive Summary & Architecture Philosophy

The **SHARE2GO** backend powers both the **SHARE2GO Mobile App** (for passengers and drivers) and the **SHARE2GO Admin Portal** (for platform operators).

It is designed as a **Modular Monolith** under **Clean Architecture** principles. This ensures:

🛡️ 1. Platform Administration Domain

Complete administrative oversight: user management, KYC approvals, ride cancellations, SOS incident center, dynamic pricing, and system audits.

`/api/v1/admin/*` `/api/v1/admin/sos/*`

👤 2. User Side (Passenger Persona)

Commuter experience: ride search with origin/destination filters, seat booking, booking history, ratings, trusted contacts, and stranger meetups.

`/api/v1/user/*` `/api/v1/stranger/*`

🚗 3. Rider Side (Driver / Host Persona)

Vehicle owner experience: driver verification KYC, vehicle fleet, ride offering, passenger booking management (accept/reject), and trip controls.

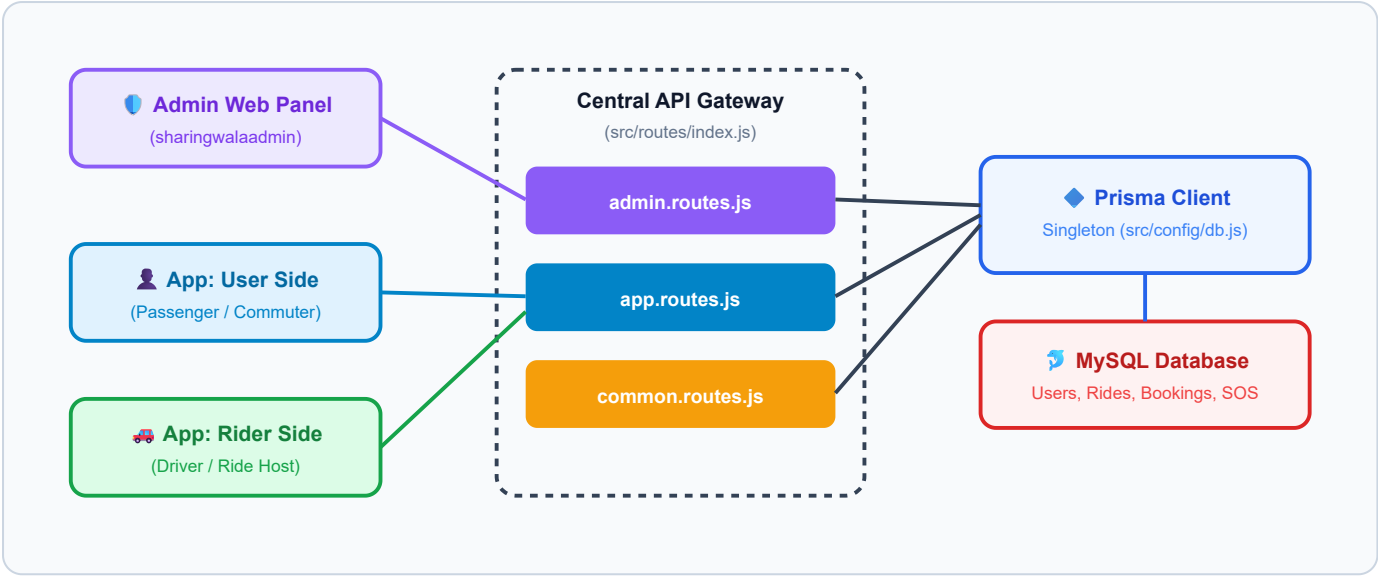
`/api/v1/rider/*`

🛠️ 4. Centralized Common Services

Shared cross-cutting platform engines: Phone OTP authentication, real-time chat, universal uploads, emergency SOS, and commute communities.

`/api/v1/auth` `/api/v1/chat` `/api/v1/upload`

2. Visual System Architecture Diagram


Admin Web Panel
(sharingwalaadmin)
App: User Side
(Passenger / Commuter)
App: Rider Side
(Driver / Ride Host)
Central API Gateway
(src/routes/index.js)
admin.routes.js
app.routes.js
common.routes.js
Prisma Client
Singleton (src/config/db.js)
MySQL Database
Users, Rides, Bookings, SOS

3. Complete Folder & File Tree

Every file has a dedicated, single responsibility following Clean Architecture:

```
D:\node_projects\SHARE2GO\ ├── client-api/ # 🎯 Reusable JavaScript Client SDK │ ├── modules/ │ │ ├── admin.api.js # 🛡️ Admin Portal methods (Stats, KYC, Rides, SOS) │ │ ├── user.api.js # 👤 Passenger methods (Search, Book, History, Ratings) │ │ ├── rider.api.js # 🚗 Driver methods (Offer, Vehicles, Passenger requests) │ │ ├── stranger.api.js # 🌟 Stranger Mode (Co-travel, Meetups, Reels) │ │ ├── auth.api.js # 🔐 OTP, JWT, Session methods │ │ └── chat.api.js # 💬 Socket.io & 1-on-1 chat methods │ ├── config.js # Base URLs & Token Storage handlers │ ├── httpClient.js # Axios instance with auto-auth interceptors │ └── index.js # Barrel export: api.admin, api.app, api.common │ ├── prisma/ │ ├── schema.prisma # Single source of truth for Models & Relations │ └── migrations/ # Versioned SQL migration history │ ├── src/ │ ├── app.js # Express middleware pipeline (cors, helmet, parsers) │ ├── server.js # Server bootstrap, Socket.io init, auto-migrate │ │ │ ├── config/ # Singletons & Infrastructure │ │ ├── db.js # Prisma Client Singleton (prevents pool leaks) │ │ ├── env.js # Validated environment variables │ │ ├── socket.js # Socket.io instance & room handlers │ │ └── firebase.js # FCM push notifications (calls & chat alerts) │ │ │ ├── middlewares/ # Cross-cutting Request Interceptors │ │ ├── auth.middleware.js # Mobile User/Driver JWT validator │ │ ├── adminAuth.middleware.js # Staff RBAC permission validator │ │ ├── validate.middleware.js # Zod schema runner for body/params/query │ │ └── error.middleware.js # Central ApiError catcher & response formatter │ │ │ ├── routes/ # 🌐 Clean Semantic Route Gateways │ │ ├── index.js # Master API Gateway mounted at /api/v1 │ │ ├── admin.routes.js # 🛡️ Platform Administration (/api/v1/admin/*) │ │ ├── app.routes.js # 📱 Mobile App (User side & Rider side) │ │ └── common.routes.js # 🛠️ Centralized Reusable Common Services │ │ │ ├── modules/ # 📦 Feature-Based Modules (Clean Architecture) │ │ ├── admin/ # Staff auth, KPIs, user management, KYC approvals │ │ ├── user/ # Passenger profile, ride search, booking, ratings │ │ ├── rider/ # Driver verification, vehicles, ride publish, trip lifecycle │ │ ├── stranger/ # Social co-travel trips, meetups, video reels │ │ ├── auth/ # Phone OTP, access/refresh token rotation │ │ ├── chat/ # Real-time messages, WebRTC call signaling │ │ ├── sos/ # Emergency SOS trigger & admin incident center │ │ ├── communities/ # Verified corporate & university commute circles │ │ ├── upload/ # Universal media upload (avatars, KYC, reels) │ │ ├── support/ # Customer support tickets & FAQs │ │ ├── common/ # Dynamic home carousel banners & app config │ │ └── health/ # Uptime, memory, and MySQL latency check │ │ │ └── utils/ # Shared Architectural Protocols │ ├── apiError.js # Controlled HTTP error class │ ├── apiResponse.js # Standardized JSON response envelope │ ├── asyncHandler.js # Controller try/catch abstraction │ └── urlHelper.js # Universal absolute media URL resolver └── ARCHITECTURE.md # Markdown documentation specification
```

4. Clean Architecture Inside Each Module

Every feature module in `src/modules/<feature>/` contains strictly 4 files:

LAYER FILE	RESPONSIBILITY	RULE
`*.routes.js`	HTTP endpoint definitions, middleware attachment	No business logic; delegates directly to controller
`*.controller.js`	Extracts params/body, calls service, returns `ApiResponse`	Always wrapped inside `asyncHandler`
`*.service.js`	Pure business logic, validation checks, database queries	Only layer that talks to `prisma` singleton

LAYER FILE	RESPONSIBILITY	RULE
`*.validation.js`	Zod input schemas for query, params, and body	Executed by `validate(schema)` middleware

5. Detailed Domain & Route Specification

🛡️ Domain 1: Platform Administration (`src/routes/admin.routes.js`)

Exclusively mounted for the Web Admin Portal (`sharingwalaadmin`):

ENDPOINT	HTTP METHOD	PURPOSE & DESCRIPTION
`/api/v1/admin/auth/login`	`POST`	Staff login with email/password, issues Admin JWT
`/api/v1/admin/dashboard/stats`	`GET`	Platform-wide KPI numbers: active users, rides, revenue
`/api/v1/admin/users`	`GET / PATCH`	Paginated commuter directory, deep search, account suspension
`/api/v1/admin/drivers/:id/verify`	`PATCH`	Approve or reject driver KYC and Driving License documents
`/api/v1/admin/vehicles/:id/verify`	`PATCH`	Approve or reject vehicle RC and Insurance papers
`/api/v1/admin/rides/:id/cancel`	`POST`	Emergency administrative cancellation of active or scheduled ride
`/api/v1/admin/sos/incidents`	`GET / PATCH`	Real-time emergency incident triage and resolution desk
`/api/v1/admin/pricing-rules`	`GET / POST`	Configure per-km rate caps, platform commissions, and fare guardrails

📱 Domain 2: Mobile App — User Side (Passenger Persona)

Passenger & Commuter screens inside the mobile Flutter app (`share2go`):

ENDPOINT	HTTP METHOD	FLUTTER SCREEN & PURPOSE
`/api/v1/user/rides/search`	`GET`	`SearchRidesScreen` : Filter by origin, destination, date, mode
`/api/v1/user/rides/:id/request-seat`	`POST`	`RideDetailsScreen` : Book seat, specify pickup note
`/api/v1/user/my-rides`	`GET`	`MyRidesScreen` : Passenger booking history and active trip tracking
`/api/v1/user/rides/:id/feedback`	`POST`	Post-ride review: 1-5 star rating and comment for driver
`/api/v1/user/trusted-contacts`	`GET / POST`	Safety preferences: Add/delete emergency trusted contacts
`/api/v1/stranger/trips`	`GET / POST`	`StrangerModeScreen` : Discover and join social co-travel trips
`/api/v1/stranger/reels`	`GET`	`StrangerReelsScreen` : Stream travel video reels

📱 Domain 3: Mobile App — Rider Side (Driver / Host Persona)

Driver & Ride Host screens inside the mobile Flutter app (`share2go`):

ENDPOINT	HTTP METHOD	FLUTTER SCREEN & PURPOSE
`/api/v1/rider/status`	`GET`	Check driver application state (PENDING, APPROVED, REJECTED)
`/api/v1/rider/apply`	`POST`	`VerificationScreen`: Submit DL number, photos, and vehicle info
`/api/v1/rider/vehicles`	`GET / POST`	Vehicle management: Add cars/bikes, upload RC and insurance
`/api/v1/rider/rides/offer`	`POST`	`OfferRideScreen`: Publish a new shared carpool or bikepool
`/api/v1/rider/my-offered-rides`	`GET`	`MyRidesScreen (Driver mode)`: View offered rides and passenger manifests
`/api/v1/rider/requests/:id/respond`	`POST`	Accept or decline passenger booking requests with push alerts
`/api/v1/rider/rides/:id/start`	`POST`	Start trip journey (notifies passengers and starts tracking)
`/api/v1/rider/rides/:id/complete`	`POST`	Complete journey (finalizes fares and unlocks passenger reviews)

🛠️ Domain 4: Centralized Common & Reusable Services

Universal infrastructure utilized across all user personas and the admin panel:

SERVICE MODULE	BASE ROUTE	CAPABILITIES PROVIDED
Auth	`/api/v1/auth`	Phone OTP verification, JWT access & refresh token rotation, session invalidation
Chat	`/api/v1/chat`	Direct 1-on-1 conversations, unread badges, WebSockets, WebRTC call signaling
Upload	`/api/v1/upload`	High-speed Multer uploads: profile avatars, vehicle documents, video reels
SOS	`/api/v1/sos`	Panic button trigger, broadcast live coordinates to trusted contacts & admin
Communities	`/api/v1/communities`	Corporate tech park and university commute circles
Support	`/api/v1/support`	Customer care inquiry ticket submission and FAQ knowledgebase
Common	`/api/v1/common`	Dynamic carousel banners, city lists, and remote app configuration
Health	`/api/v1/health`	Server uptime, memory utilization, and MySQL database ping latency

6. Reusable JavaScript Client SDK (`client-api/`)

Located at `client-api/` in the project root, this standalone SDK provides ready-to-use methods for any web or node application:

```
// Example Usage of the Centralized Client SDK: import api from './client-api/index.js'; // 1. Platform Administration const stats = await api.admin.getDashboardStats(); const drivers = await api.admin.getDrivers({ status: 'PENDING' }); // 2. App - User Side (Passenger) const rides = await api.app.user.searchRides({ origin: 'Hitech City', destination: 'Gachibowli' }); const booking = await api.app.user.requestSeat(rideId, { seats: 1 }); // 3. App - Rider Side (Driver) const offered = await api.app.rider.offerRide(rideData); const response = await api.app.rider.respondBooking(bookingId, { action: 'ACCEPT' }); // 4. Centralized Common const messages = await api.common.chat.getMessages(conversationId);
```

7. Summary: 5 Golden Rules for Developers

RULE	REQUIREMENT	WHY IT MATTERS
1. Database Singleton	Always import `prisma` from `src/config/db.js`	Prevents MySQL connection pool exhaustion in dev hot-reloads.
2. Error Handling	Throw `new ApiError(statusCode, message)`	Ensures clean JSON error responses without crashing the server.

RULE	REQUIREMENT	WHY IT MATTERS
3. Response Standard	Return `new ApiResponse(statusCode, data, message)`	Provides a predictable contract: `{ success, statusCode, message, data }`.
4. Async Safety	Wrap all controllers in `asyncHandler(...)`	Eliminates boilerplate `try/catch` blocks across the codebase.
5. Validation	Define Zod schemas in `*.validation.js`	Guarantees strictly typed input validation before hitting services.